ASSIGNMENT 5

Instructions for Students

- Create a **separate HTML and CSS file for each question**.
- Do **not use JavaScript or any external framework**.
- Focus on **semantic HTML** and **clean, well-indented CSS**.
- Use **comments** in your CSS to explain the logic where necessary.

---

1. Create a **personal portfolio homepage** using HTML and CSS.
Requirements:

- The header should contain your name and navigation links (Home, About, Contact).
- Use **`position: fixed`** for the header so it stays at the top when scrolling.
- The main section should have a short bio paragraph and a profile image.
- Use **different CSS units** (**`px`, `em`, `%`, `vh`, `rem`**) across elements.
- Use **custom background color and text color combinations** that maintain contrast.
- The footer should always stay at the bottom of the page (use `flexbox` or positioning).

2. Design a **multi-section informational webpage** (for example: “About a City” or “Favorite Destination”).
Requirements:

- Include a **`<header>`**, **`<nav>`**, **`<main>`**, and **`<footer>`**.
- Each section in **`<main>`** should have a **full-width background color or image**.
- Apply a **linear gradient** in one section and a **radial gradient** in another.
- Use **padding, margins, and border** to create spacing and define clear sections.
- Headings must use **different font sizes and weights** using CSS.

- At least two sections must demonstrate **background-attachment: fixed;** (parallax-like effect).

3. Build a **3-column "Team Members" layout** showing profile cards for three people.
Requirements:

- Use **`display: flex`** or **`display: grid`** to align cards in one row.
- Each card should contain a name, role, and short description.
- Use the **Box Model** to create uniform padding and margins.
- Add **borders, rounded corners, and shadow effects** to each card.
- Highlight one card (e.g., Team Lead) with a different background color and border style.
- The layout must **automatically adjust to a single column** when viewed on a screen smaller than 600px (responsive layout).

4. Create a **long article webpage** with the following structure:

- Header → Article Title
- Subheading → Author name and publication date
- Body → At least 5 paragraphs of dummy text
  Use CSS to:
- Add consistent spacing between paragraphs using margins.
- Style the first letter of each paragraph with a larger font using **`::first-letter`**.
- Make the first line of each paragraph bold using **`::first-line`**.
- Highlight key words using **`<mark>`** and style it with custom background color.
- Add a left border and background color to emphasize the blockquote section.

5. Design a **pricing table** with three pricing plans: Basic, Standard, and Premium.
Requirements:

- Use **flexbox or grid** to create three equal columns.
- Add a **hover effect** on each plan that slightly enlarges the box and changes background color.
- Use CSS transitions for smooth hover animations.
- Each plan should have:
    - A heading (plan name)
    - A list of features
    - A price section
    - A button (styled link using <a>).
- Use **different border styles** or colors to highlight the most popular plan.
- All boxes should maintain equal height even when text varies.

6. Create a **CSS-only photo gallery** layout with at least 6 images.
Requirements:

- Use **`flexbox`** or **`grid`** to display images in multiple columns.
- When hovered, each image should:
    - Slightly scale up (**`transform: scale(1.1)`**)
    - Show a caption overlay using **`position: absolute`** and **`opacity`**.
- Ensure that the gallery is responsive and adjusts image count per row based on screen width.
- Use CSS **`box-shadow`** and **`border-radius`** to make images look clean and modern.

7. Design a **contact page** using a semantic **`<form>`**.
Requirements:

- Include fields for Name, Email, Subject, and Message.
- Center the form vertically and horizontally using CSS layout techniques.
- Style all inputs with padding, border-radius, and subtle shadows.
- Change the border color of an input when it is focused (**`:focus`** selector).
- Use **`background-color`** and **`color`** CSS properties to give a consistent theme.
- Add spacing between fields using margins and test layout alignment without using tables.

8. Create a **dashboard layout** using only HTML and CSS.
Requirements:

- Header section at the top (fixed position).
- Sidebar on the left containing navigation links.
- Main content area on the right with at least two content boxes.
- Use **positioning and flexbox** to structure the layout.
- Ensure the sidebar height adjusts to full screen height.
- The content boxes should have shadows and be evenly spaced.
- On smaller screens, the sidebar should stack above the content.

9. Build a **CSS layout that mimics a newspaper or magazine column structure**.
Requirements:

- Create three vertical text columns using **`column-count`** and **`column-gap`**.
- Add a heading that spans across all columns using **`column-span: all;`**.

- Apply custom font styles and justify all text.
- Include at least one image aligned to the left using **`float`**.
- Ensure the layout remains clean and readable when resized.
- Add background color to the page and padding around the main content area.

10. Design a **complete webpage layout** demonstrating mastery of the CSS Box Model and positioning.
Requirements:

- Header with navigation (sticky at top).
- Hero section with a large background image and centered welcome text.
- About section with two columns (text + image).
- Services section with three boxes in a row.
- Contact section with a simple form.
- Footer with copyright text.
- Use at least:
    - **`margin`**, **`padding`**, **`border`**, and **`outline`** properties.
    - Relative, absolute, and fixed positioning in different sections.
    - **`flexbox`** or **`grid`** for responsive alignment.
    - **`rgba()`** colors and gradients.
- Ensure all elements are visually aligned, balanced, and responsive.